



Database Management Systems

Week 2, Day 6

MySQL—A RDBMS

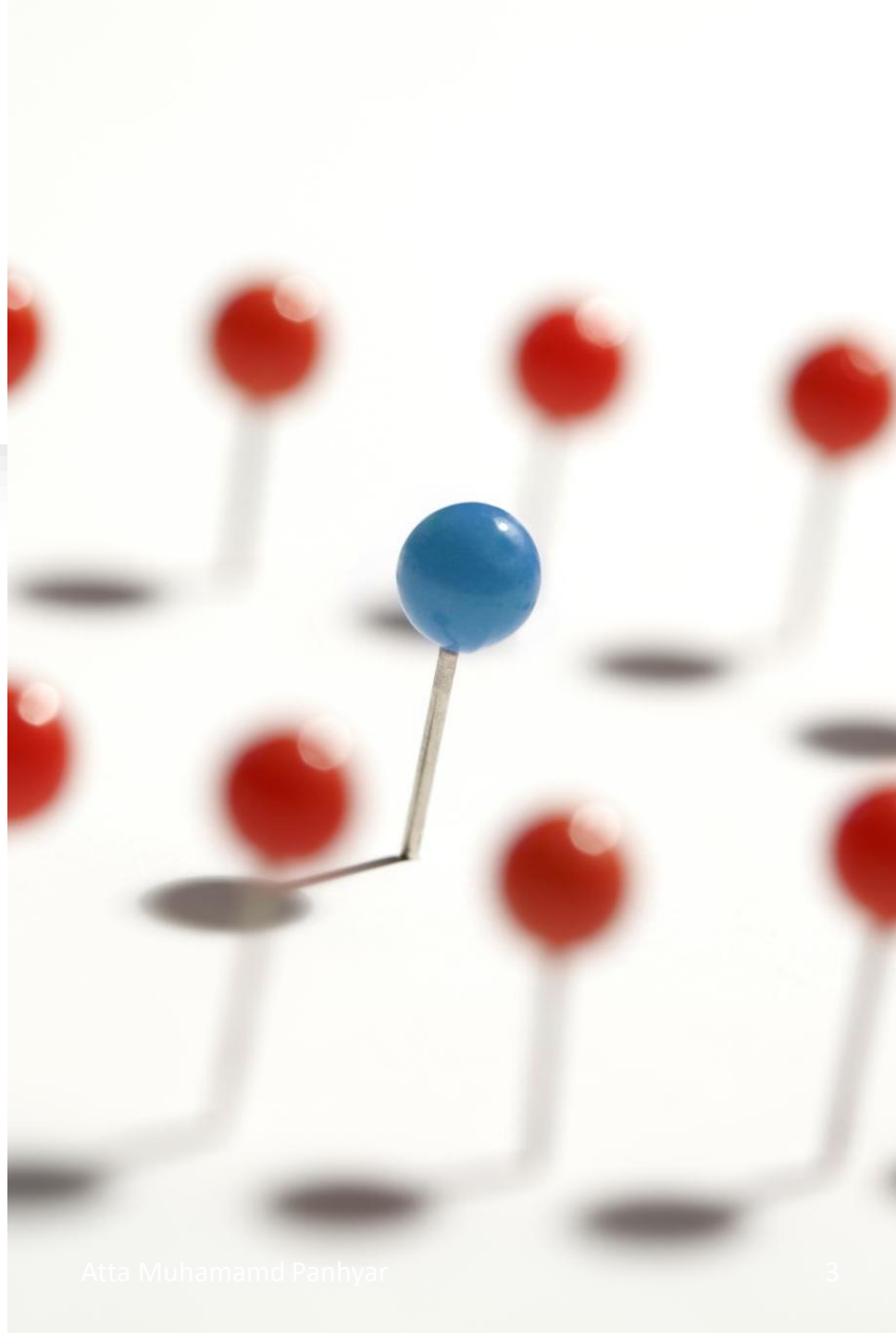
Atta Muhammad

+92-333-1971110

attapanhyar@quest.edu.pk

RECAP

- Having Clause
- Inner Join
- LEFT/RIGHT JOIN



EXISTS Operator

- The **EXISTS** operator is used to test for the existence of any record in a subquery.
- The **EXISTS** operator returns **TRUE** if the subquery returns one or more records.

EXISTS Example

```
1 • SELECT SupplierName FROM Suppliers
2 WHERE EXISTS( SELECT ProductName FROM Products
3 WHERE Products.SupplierID = Suppliers.supplierID
4 AND Price < 20);
```

Alt Grid |   Filter Rows: | Export:  | Wrap Cell Content:  | Fetch rows: 

SupplierName

Exotic Liquid

CASE

- The **CASE** statement goes through conditions and returns a value when the first condition is met (like an if-then-else statement).
- So, once a condition is true, it will stop reading and return the result.
- If no conditions are true, it returns the value in the **ELSE** clause.

CASE –Example

```
1 • SELECT OrderID, Quantity,  
2 CASE  
3     WHEN Quantity > 30 THEN 'The quantity is greater than 30'  
4     WHEN Quantity = 30 THEN 'The quantity is 30'  
5     ELSE 'The quantity is under 30'  
6 END AS QuantityText  
7 FROM OrderDetails;
```

ilt Grid   Filter Rows: Export:  Wrap Cell Content:  Fetch rows: 		
OrderID	Quantity	QuantityText
10248	12	The quantity is under ...
10248	10	The quantity is under ...

CASE –Example

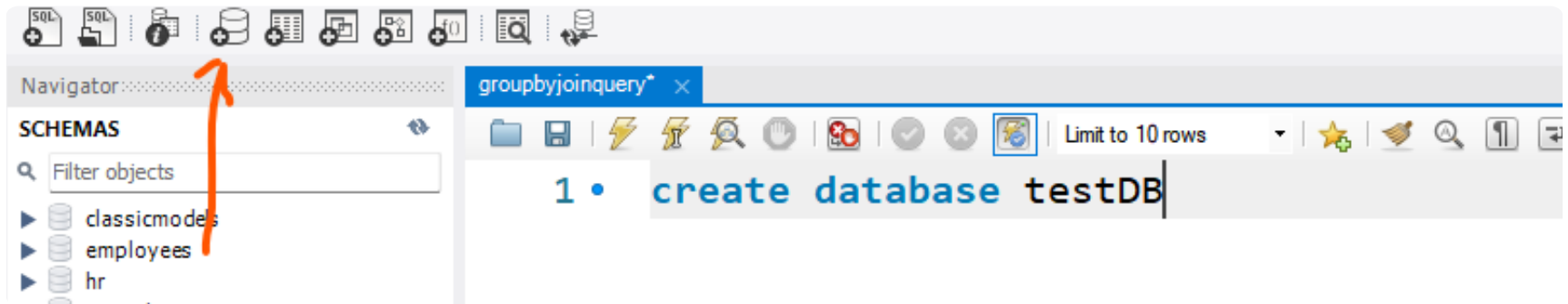
```
1 • SELECT CustomerName, City, Country
2   FROM Customers
3   ORDER BY
4   (CASE
5       WHEN City IS NULL THEN Country
6       ELSE City
7   END);
```

Alt Grid  Filter Rows: Export:  Wrap Cell Content:  Fetch rows: 		
CustomerName	City	Country
Drachenblut Delikates...	Aachen	Germany
Rattlesnake Canyon G...	Albuquerque	USA

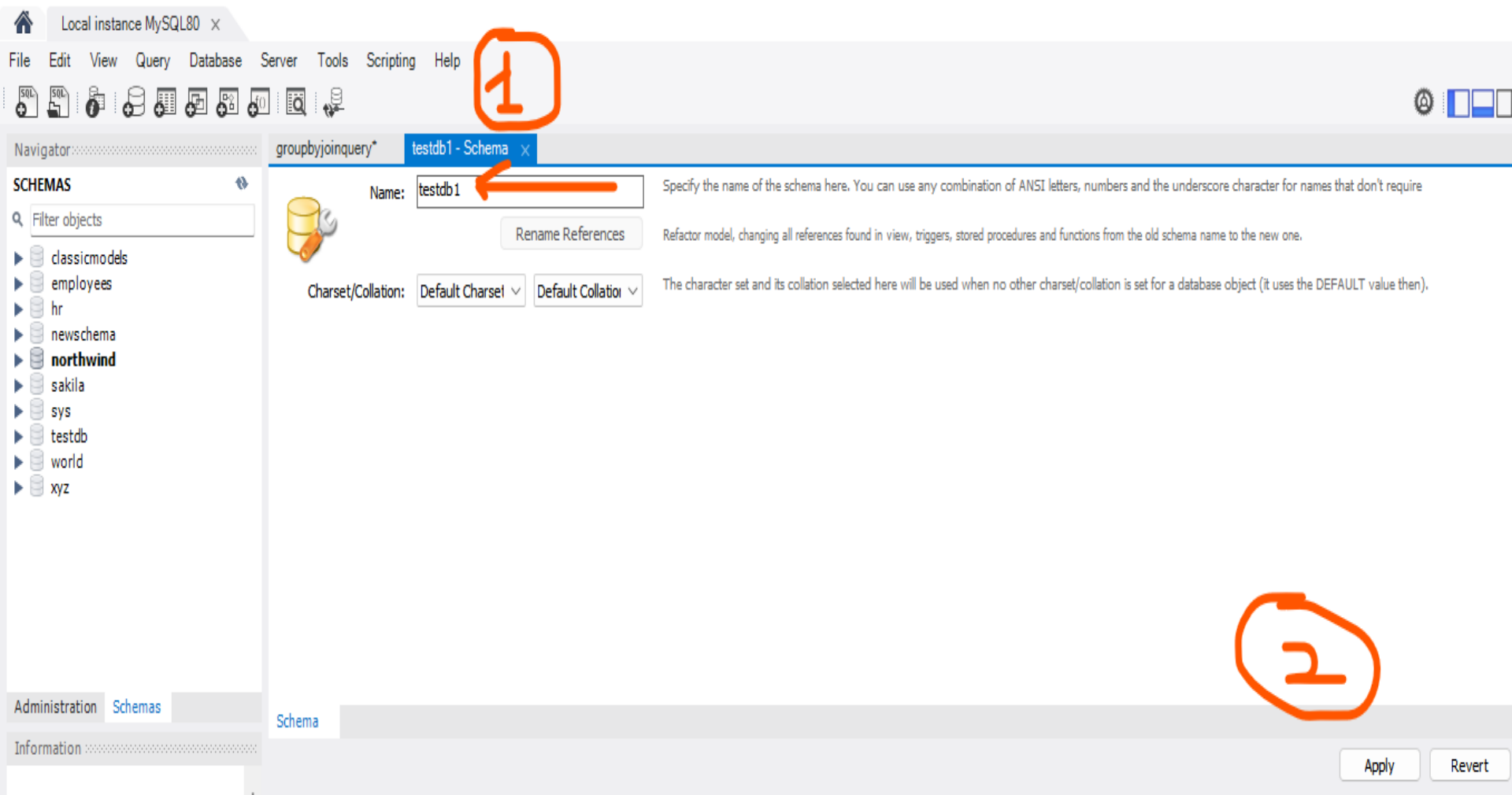
Create a Database

- The **CREATE DATABASE** statement is used to create a new SQL database.

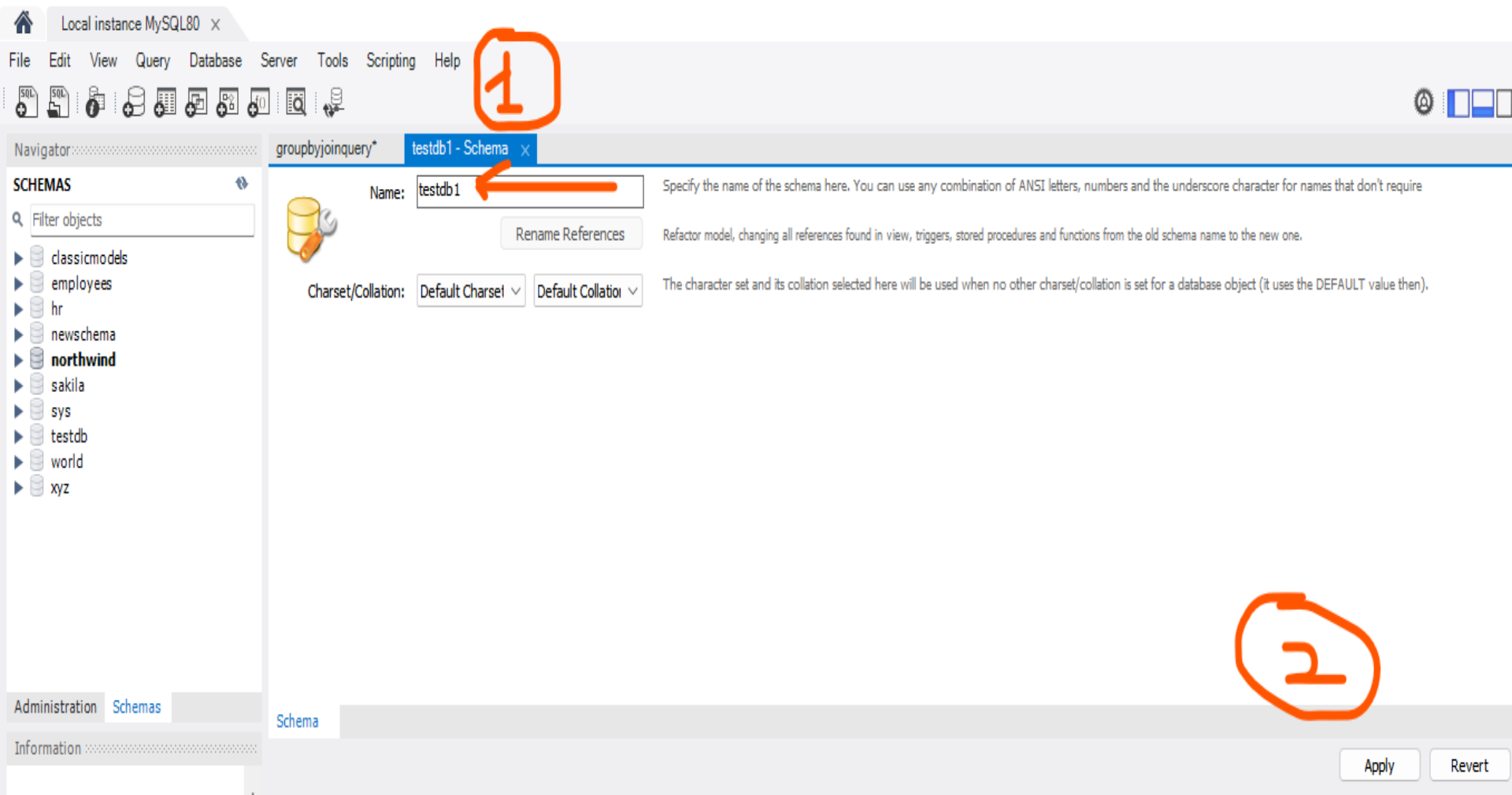
```
1 • create database testDB
```



CREATE a DATABASE



DATABASE



DATABASE

Review SQL Script

Apply SQL Script

Review the SQL Script to be Applied on the Database

Online DDL

Algorithm:

Default



Lock Type:

Default



```
1 CREATE SCHEMA `testdb1` ;
```

```
2
```



Navigator

SCHEMAS

Filter objects

- classicmodels
- employees
- hr
- newschema
- northwind**
- sakila
- sys
- testdb
- testdb1
- world
- xyz

Administration Schemas

Information

Schema: testdb1

Object Info Session

groupbyjoinquery* testdb1 - Schema testdb1 x

Info Tables Columns Indexes Triggers Views Stored Procedures Functions Grants Events

Name	Engine	Version	Row Format	Rows	Avg Row Length	Data Length

Count: 0

Maintenance >

Output

Action Output

	#	Time	Action	Message
✓	10	18:44:37	create database testDB	1 row(s) affected
✗	11	18:47:56	Apply changes to testDB	Selected name conflicts with
✗	12	18:48:22	Apply changes to testdb1	
✓	13	18:52:12	Apply changes to testdb1	Changes applied

Review SQL Script

Apply SQL Script

Review the SQL Script to be Applied on the Database

Online DDL

Algorithm:

Default

Lock Type:

Default

```

1  CREATE TABLE `testdb1`.`tblstudents` (
2      `studentid` INT NOT NULL AUTO_INCREMENT,
3      `studentName` VARCHAR(45) NOT NULL,
4      `tblStudentscol` VARCHAR(45) NULL,
5      `stdfathername` VARCHAR(45) NULL,
6      `stdage` INT NULL,
7      `stdCNIC` VARCHAR(45) NULL,
8      `tblStudentscol1` VARCHAR(45) NULL,
9      PRIMARY KEY (`studentid`))
10 COMMENT = 'This table is create for adding students information';
11

```


[Review SQL Script](#)**[Apply SQL Script](#)**

Applying SQL script to the database

The following tasks will now be executed. Please monitor the execution. Press Show Logs to see the execution logs.

☒ Execute SQL Statements

SQL script was successfully applied to the database.

11/25/2024

[Show Logs](#)[Back](#)[Finish](#)[Cancel](#)

DROP DATABASE Statement



ALTER TABLE Statement

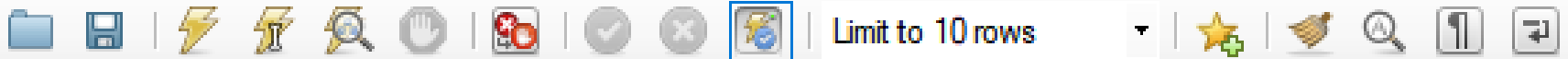
- The **ALTER TABLE** statement is used to add, delete, or modify columns in an existing table.
- The **ALTER TABLE** statement is also used to add and drop various constraints on an existing table

```
1  ALTER TABLE Customers
2  ADD Email varchar(255);
```

ALTER TABLE –Example

ALTER TABLE - DROP COLUMN

- To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):



```
1 • ALTER TABLE Customers
2 DROP COLUMN Email;
```

ALTER TABLE –DROP COLUMN

ALTER TABLE - MODIFY COLUMN

- To change the data type of a column in a table, use the following syntax



```
1 • ALTER TABLE customers
2   ADD DateOfBirth date;
3   -- Run these commands separetly
4 • ALTER TABLE customers
5   MODIFY COLUMN DateOfBirth year;
```

MODIFY

Constraints

- SQL constraints are used to specify rules for data in a table.
- Constraints can be specified when the table is created with the **CREATE TABLE** statement, or after the table is created with the **ALTER TABLE** statement.

Constraints

- Constraints are used to limit the type of data that can go into a table.
- This ensures the accuracy and reliability of the data in the table.
- If there is any violation between the constraint and the data action, the action is aborted.

Constraints

- Constraints can be **column level** or **table level**.
- **Column level** constraints apply to a column, and **table level** constraints apply to the whole table.

Common Constraints

- **NOT NULL** - Ensures that a column cannot have a NULL value
- **UNIQUE** - Ensures that all values in a column are different
- **PRIMARY KEY** - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- **FOREIGN KEY** - Prevents actions that would destroy links between tables
- **CHECK** - Ensures that the values in a column satisfies a specific condition
- **DEFAULT** - Sets a default value for a column if no value is specified
- **CREATE INDEX** - Used to create and retrieve data from the database very quickly

NOT NULL Constraint

- By default, a column can hold **NULL** values.
- The **NOT NULL** constraint enforces a column to NOT accept NULL values.

```
• CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255) NOT NULL,  
    Age int  
);
```

Constraints on ALTER TABLE

```
ALTER TABLE Persons  
MODIFY Age int NOT NULL;
```

UNIQUE Constraint

- The **UNIQUE** constraint ensures that all values in a column are different.
- A **PRIMARY KEY** constraint automatically has a **UNIQUE** constraint.

UNIQUE Constraint on CREATE TABLE

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    UNIQUE (ID)  
);
```


Graphically...

SQL File 3* northwind tblTest - Table x

 Table Name: Schema: **northwind**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
 stdid	INT	☒	☒	☒	☐	☐	☐	☐	☐	

Column Name: Data Type:

Charset/Collation:

Comments:

Default:

Storage: ☐ Virtual ☐ Stored

☒ Primary Key ☒ Not Null ☒ Unique

☐ Binary ☐ Unsigned ☐ Zero Fill

☐ Auto Increment ☐ Generated

Columns Indexes Foreign Keys Triggers Partitioning Options

PRIMARY KEY Constraint

- The **PRIMARY KEY** constraint uniquely identifies each record in a table.
- Primary keys must contain **UNIQUE** values and cannot contain **NULL** values.
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields)

FOREIGN KEY Constraint

- The **FOREIGN KEY** constraint is used to prevent actions that would destroy links between tables.
- A **FOREIGN KEY** is a field (or collection of fields) in one table, that refers to the **PRIMARY KEY** in another table.

FOREIGN KEY

```
CREATE TABLE Orders (  
    OrderID int NOT NULL,  
    OrderNumber int NOT NULL,  
    PersonID int,  
    PRIMARY KEY (OrderID),  
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
);
```

Question(s)
